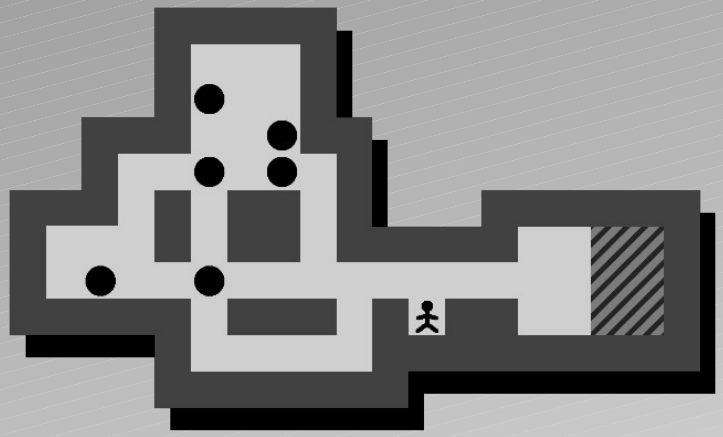
Jeu de Sokoban

Recherche de solutions optimales

Mémoire réalisé en vue de l'obtention du grade de Licencié en Informatique sous la direction du Professeur **Véronique Bruyère**

Michaël Hoste



ACADÉMIE UNIVERSITAIRE WALLONIE-BRUXELLES

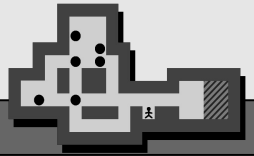
Juin 2008

UMH

Université de Mons-Hainaut | Faculté des Sciences | Institut d'Informatique

> Structure de l'exposé

- **Sokoban**
  - Présentation
  - Règles
- **Résolution**
  - État de l'art
  - Arbre de recherche
- **Parcours**
  - en largeur
  - en profondeur
  - A*
  - IDA*
- **Estimation**
- **Deadlock**
  - À une caisse
  - À plusieurs caisses
- **Pénalité**
  - Définition
  - Recherche
- **Pré-traitement & Post-traitement**
- **Conclusion**

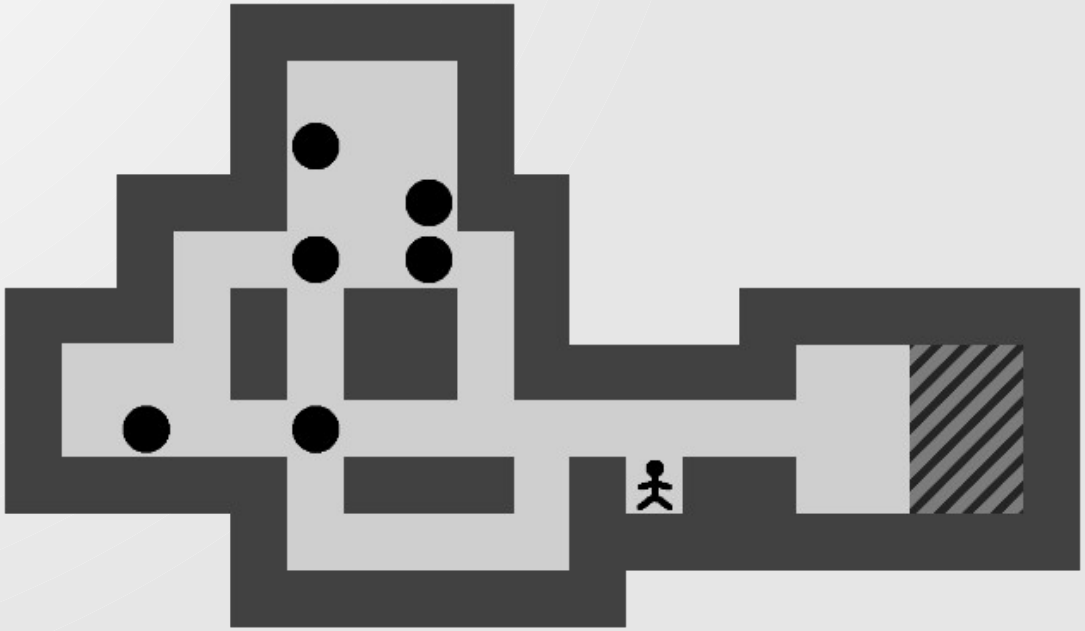


> Présentation

- **Sokoban est :**
  - **Un jeu vidéo** japonais de réflexion datant des années 80.
  - **Un problème de planification** de type « Single-Agent ».
  - Un défi pour les domaines de l'**intelligence artificielle** et de la **complexité**.
    - Le problème de décision relatif à Sokoban est **PSPACE-complet**

> Règles (1)

- Sokoban est composé de **niveaux** :



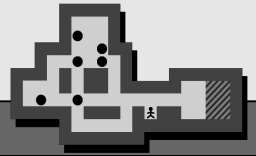
- Chaque niveau correspond aux **positions initiales** des **objets fixes** (murs, goals) et **mobiles** (caisses, pousseur).

> Règles (2)

- **Le joueur dirige le pousseur**. Celui-ci peut se déplacer **sur ses 4 cases voisines** (haut, bas, droite et gauche).

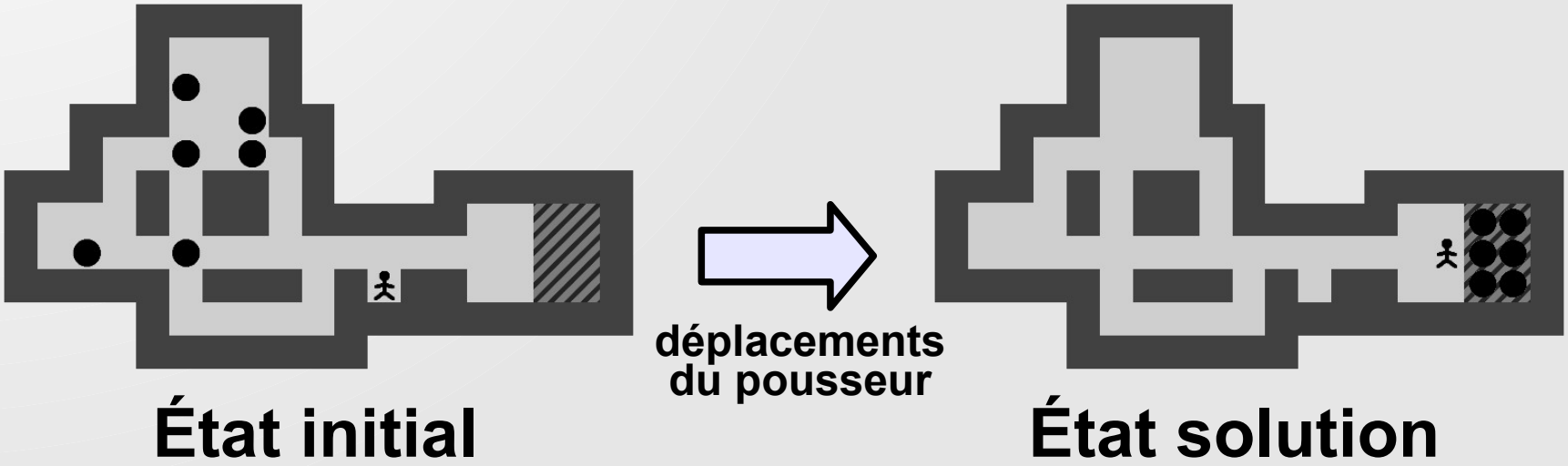
- **Le but du jeu est de réussir à positionner toutes les caisses sur des goals**.

- Les principales **contraintes** sont les suivantes :
  - Le pousseur ne peut **que pousser des caisses**, **pas les tirer**.
  - Le pousseur ne peut **pousser qu'une caisse à la fois**. Si deux caisses sont voisines sur un axe, il ne pourra pas les pousser sur cet axe.



> Règles (3)

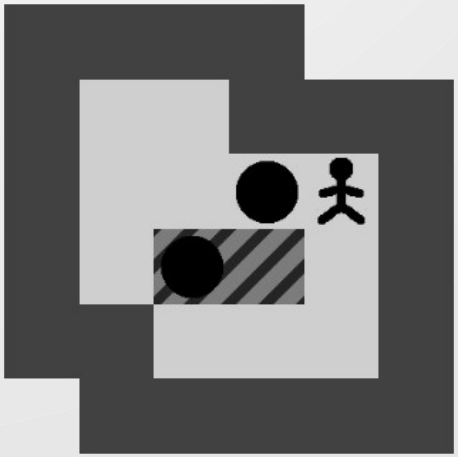
- **État** : position des éléments mobiles (pousseur et caisses) après un certain nombre de déplacements du pousseur dans le niveau.

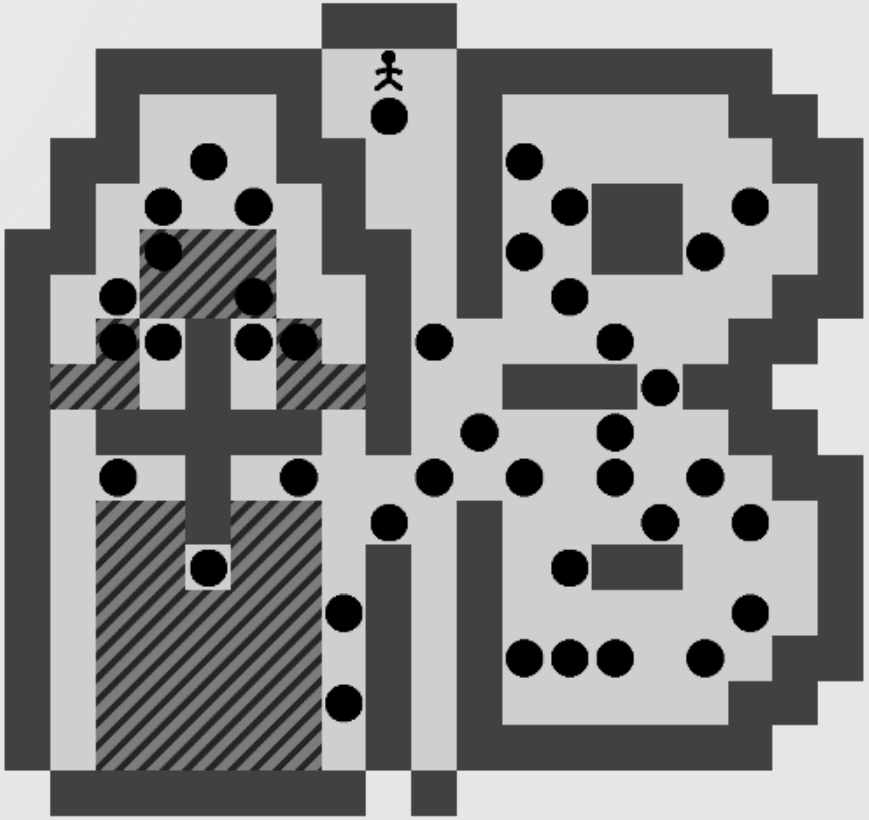


- **Solution** : succession des déplacements du pousseur permettant de positionner toutes les caisses sur des goals.

> Règles (4)

- **Les niveaux existent par milliers et sont de tailles et de difficultés très variables.**





> Structure de l'exposé

- **Sokoban**
  - Présentation
  - Règles
- **Résolution**
  - État de l'art
  - Arbre de recherche
- **Parcours**
  - en largeur
  - en profondeur
  - A*
  - IDA*
- **Estimation**
- **Deadlock**
  - À une caisse
  - À plusieurs caisses
- **Pénalité**
  - Définition
  - Recherche
- **Pré-traitement & Post-traitement**
- **Conclusion**

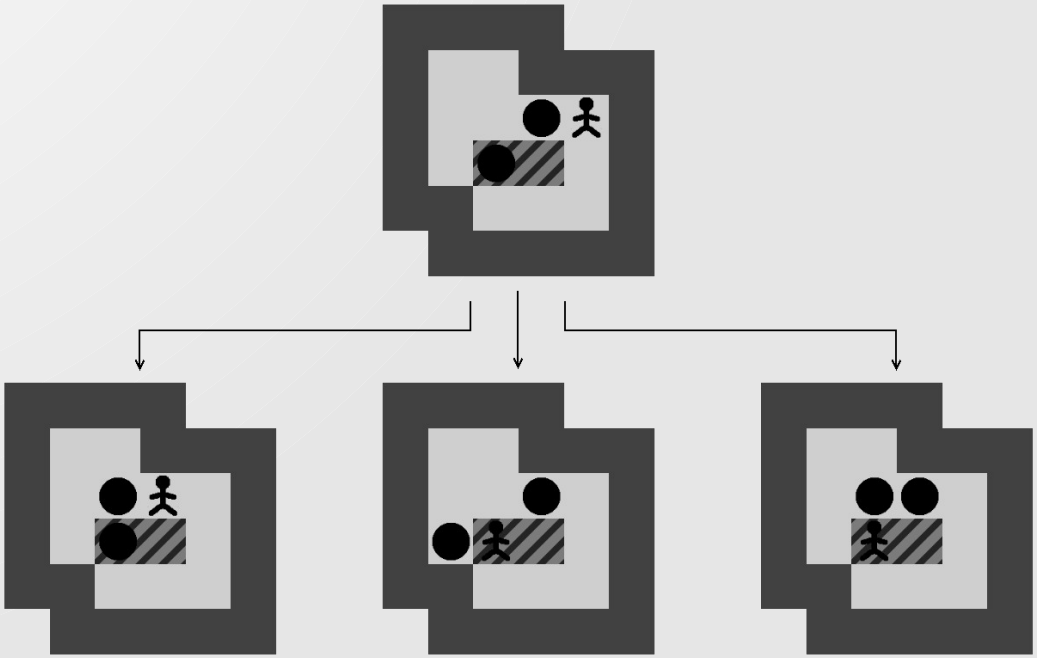
> État de l'art

- Plusieurs recherches ont déjà été effectuées et ont conduit à la conception des solveurs suivants :
  - **Rolling Stone** : Université d'Alberta (Canada), très documenté, solutions optimales, programmé en C, **57** niveaux résolus.
  - **Talking Stone** : Université de Liège, approche « Multi-agent », solutions non-optimales, programmé en Scheme, **9** niveaux résolus.
  - **Talking Stone (2)** : Université de Liège, approche « Multi-agent » avec ordonnancement des goals, solutions non-optimales, programmé en Scheme, **54** niveaux résolus.
  - **Sokoban Automatic Solver** : Ken'ichiro Takahashi, non documenté, solutions non-optimales, **86** niveaux résolus.

L'efficacité des solveurs est testée sur un même ensemble de 90 niveaux qui sert de référence

> Arbre de recherche (1)

- Le point commun entre tous ces solveurs est l'**arbre de recherche**.



Chaque **transition** dans l'arbre correspond à une **succession de déplacements suivie d'une poussée**

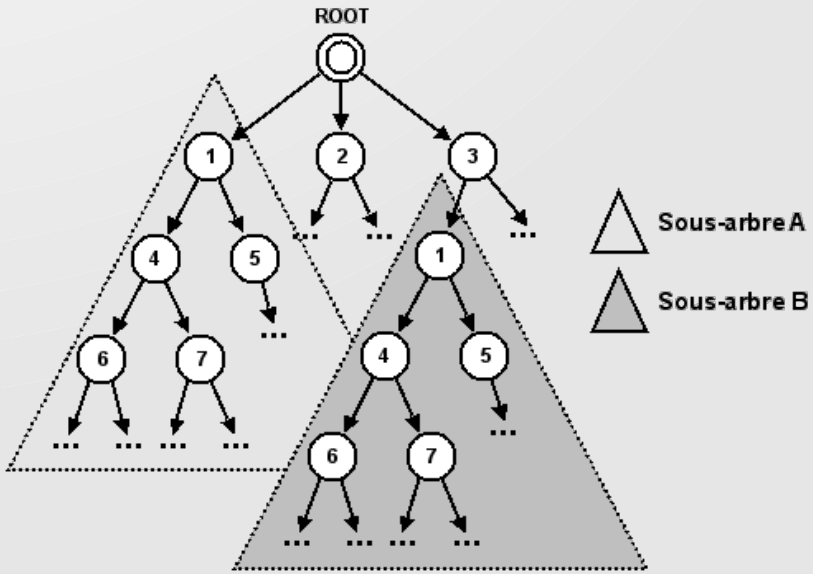
> Arbre de recherche (2)

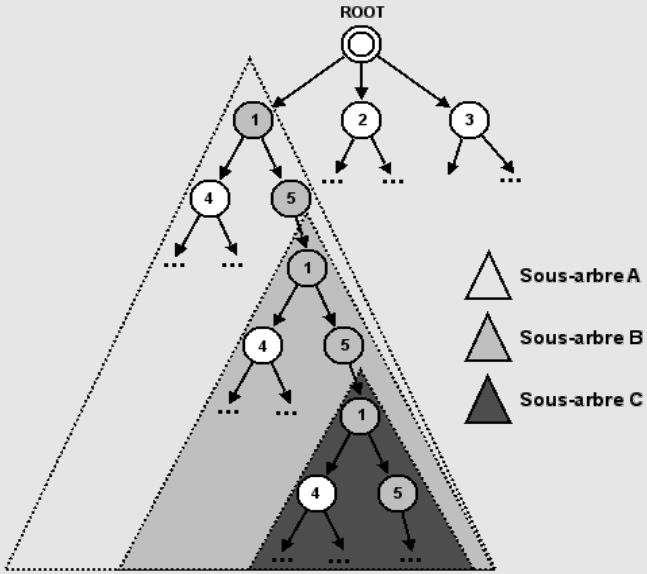
- La construction de l'arbre de recherche a **pour objectif de trouver un état solution** à partir de l'état initial (la racine).

- Lorsqu'un état solution est trouvé, il suffit de **remonter dans l'arbre en évoluant de père en père jusqu'à la racine pour déduire la solution** du niveau.

> Arbre de recherche (3)

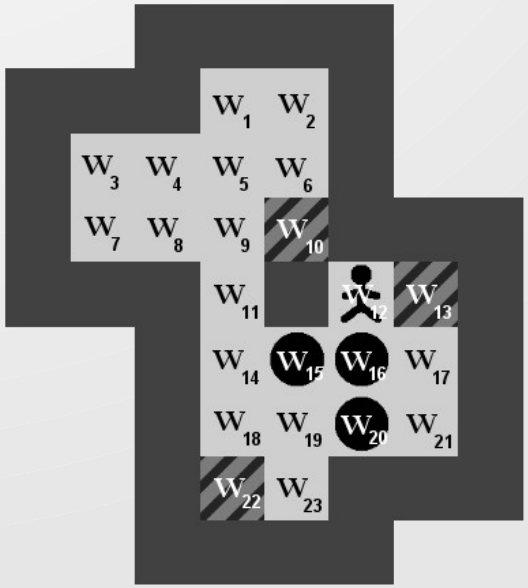
- La **construction d'un arbre de recherche** nécessite :
  - **Une liste d'attente** : elle contient les futurs états à ajouter dans l'arbre. Le parcours de l'arbre dépendra de la gestion de la liste.
  - **Une table de hachage** : elle contient tous les états compris dans l'arbre afin de détecter rapidement les doublons





> Arbre de recherche (4)

- Pour **réduire au maximum l'occupation mémoire de l'arbre de recherche**, le concept de **zone** a été créé pour stocker un sous-ensemble de positions “internes” sous une forme binaire.



Dans cet exemple, la **zone des caisses** de l'état comprend le sous-ensemble de positions $\{W_{15}, W_{16}, W_{20}\}$

Les bits 15, 16 et 20 de l'entier correspondant seront donc assignés à 1 :

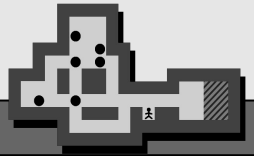
00000000000000**11**000**1**000 (000000000)

L'entier correspondant est **200704**.

Comme des états ne sont représentés que par deux zones (caisses et pousseur), il est possible de **représenter un état avec seulement deux entiers**.

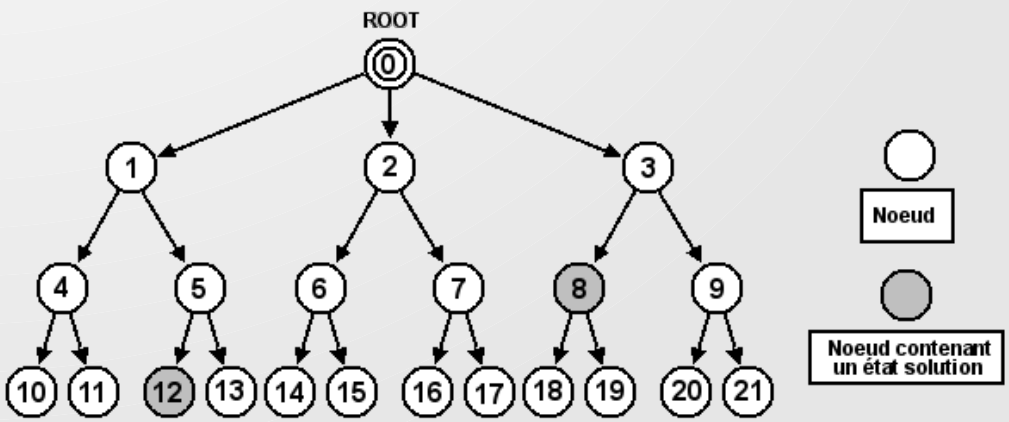
> Structure de l'exposé

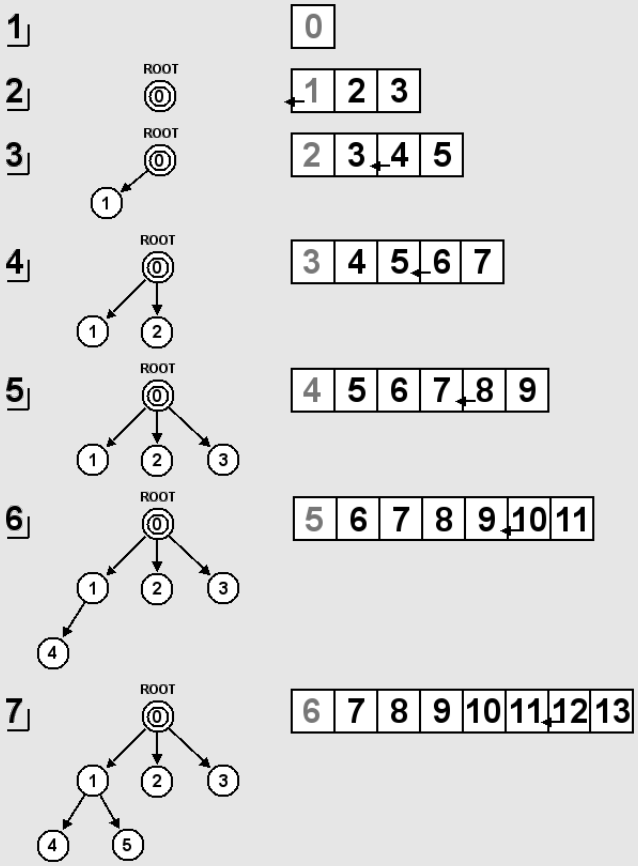
- **Sokoban**
  - Présentation
  - Règles
- **Résolution**
  - État de l'art
  - Arbre de recherche
- **Parcours**
  - en largeur
  - en profondeur
  - A*
  - IDA*
- **Estimation**
- **Deadlock**
  - À une caisse
  - À plusieurs caisses
- **Pénalité**
  - Définition
  - Recherche
- **Pré-traitement & Post-traitement**
- **Conclusion**



> Parcours en largeur

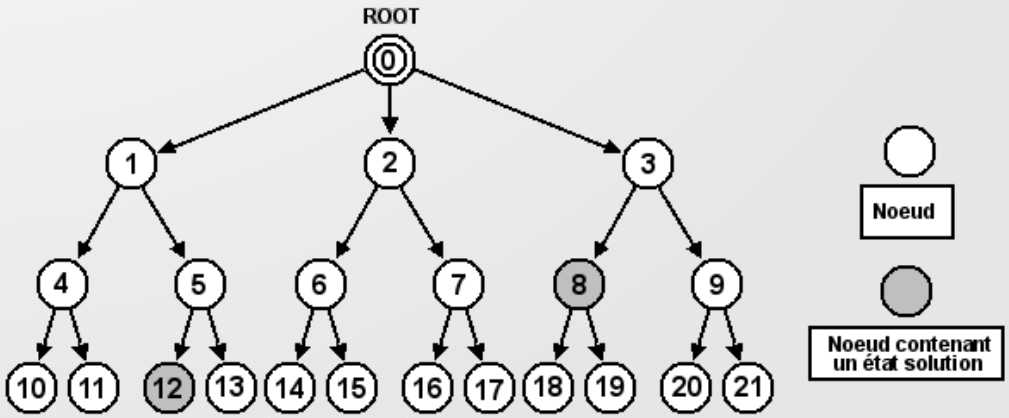
- Traite en priorité les noeuds voisins situés sur un même niveau de l'arbre de recherche. **La liste d'attente des états est une file**. Les solutions trouvées sont **optimales**.

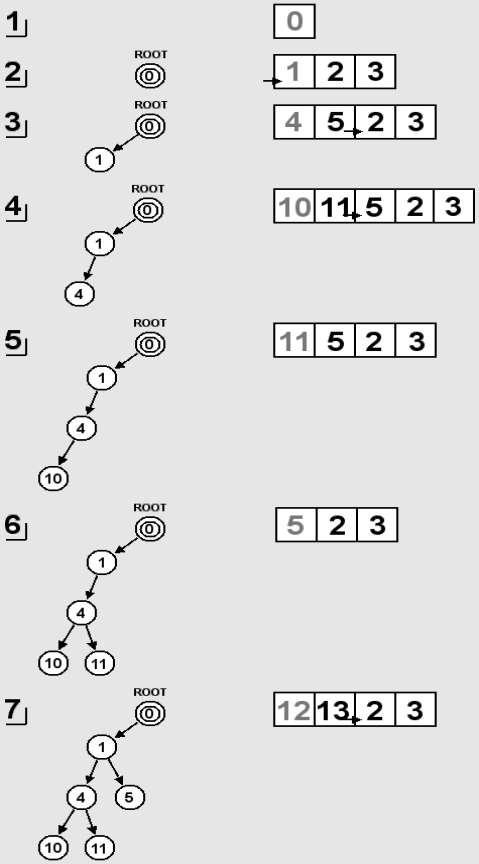




> Parcours en profondeur

- Traite en priorité les noeuds situés en profondeur dans l'arbre de recherche. **La liste d'attente des états est une pile**. Les solutions trouvées sont **non-optimales**.





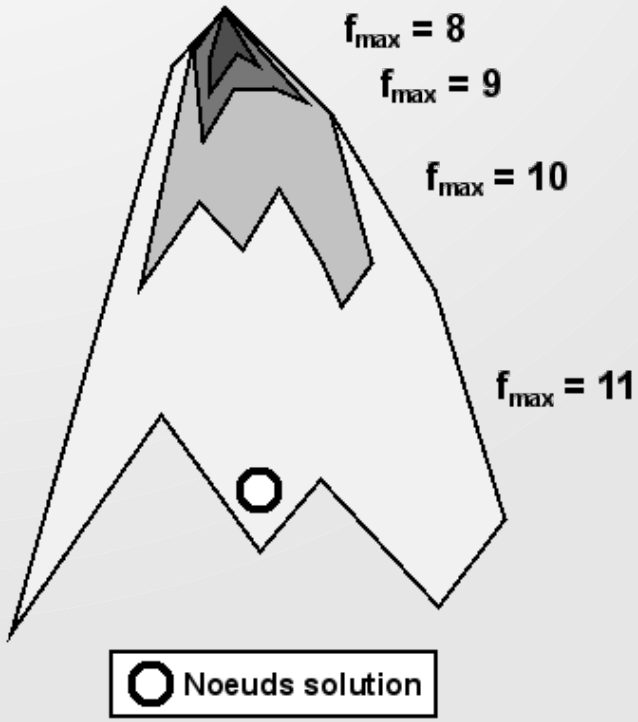
> Parcours A*

- **Parcours informé** dans lequel nous définissons le coût d'un noeud de l'arbre de recherche par **f(n) = g(n) + h(n)** où :
  - **g(n) est la distance déjà parcourue** par le noeud n. C'est-à-dire le nombre de poussées déjà effectuées entre le noeud racine et le noeud n.
  - **h(n) est l'estimation de la distance restante** à parcourir pour trouver un noeud solution à partir du noeud n.

Le parcours A* permet d'explorer en premiers les noeuds les plus prometteurs. Si l'estimation h(n) est minorante du coût réel, la solution trouvée sera optimale.

> Parcours IDA*

- **Utilisation du parcours A*** avec rejet de tous les noeuds pour lesquels **f(n) > f_{max}** où **f_{max} est une valeur fixée** qui augmente à chaque itération infructueuse du parcours sur un niveau.



— Si une itération avec **f_{max} = 10** ne retourne pas de solution, on peut affirmer que la solution optimale du niveau contiendra plus de 10 poussées.

— La prochaine itération du parcours commencera avec la valeur **f_{max} = 11**.

> Structure de l'exposé

- **Sokoban**
  - Présentation
  - Règles
- **Résolution**
  - État de l'art
  - Arbre de recherche
- **Parcours**
  - en largeur
  - en profondeur
  - A*
  - IDA*
- **Estimation**
- **Deadlock**
  - À une caisse
  - À plusieurs caisses
- **Pénalité**
  - Définition
  - Recherche
- **Pré-traitement & Post-traitement**
- **Conclusion**

> Calculer h(n) (1)

- Lors du parcours A*, le plus grand challenge est de réussir à **créer une fonction h(n)** permettant de trouver **l'estimation la plus juste possible** du nombre de poussées restantes.

- **Plus h(n) sera précis et plus le parcours sera efficace.**

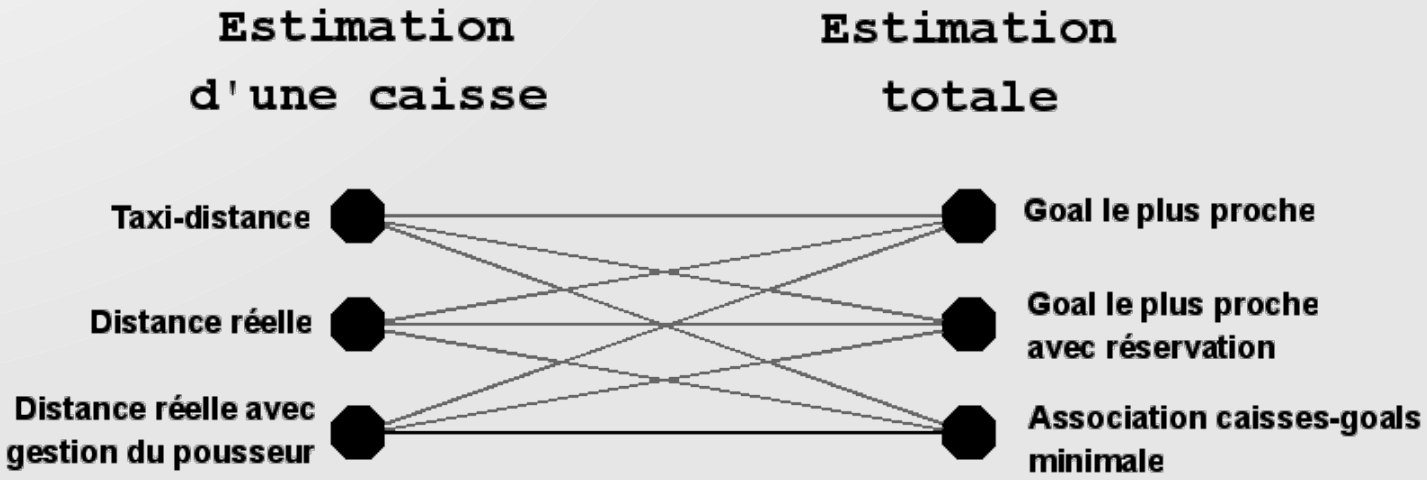
- **Si h(n) est parfait**, c'est-à-dire qu'il correspond toujours au nombre de poussées entre le noeud n et le noeud solution, **alors le parcours ira directement du noeud racine au noeud solution** sans s'égarer.

La valeur h(n) doit impérativement minorer le nombre réel de poussées restantes pour que le parcours A* indique la solution optimale.

> Calculer h(n) (2)

- Le calcul de h(n) est un processus qui s'effectue en **deux étapes** :

  1. Estimer le nombre de poussées requises pour positionner **chacune des caisses vers chacun des goals**.
  2. Sur base des résultats de (1), trouver la **combinaison caisses-goals** permettant de minimiser le nombre total de poussées.



> Structure de l'exposé

- **Sokoban**
  - Présentation
  - Règles
- **Résolution**
  - État de l'art
  - Arbre de recherche
- **Parcours**
  - en largeur
  - en profondeur
  - A*
  - IDA*
- **Estimation**
- **Deadlock**
  - À une caisse
  - À plusieurs caisses
- **Pénalité**
  - Définition
  - Recherche
- **Pré-traitement & Post-traitement**
- **Conclusion**

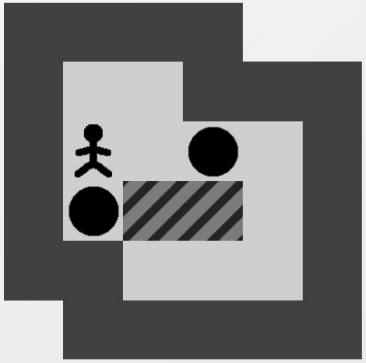
> Définition

- Un **deadlock** est un état de l'arbre de recherche à partir duquel aucun état solution ne pourra être trouvé.

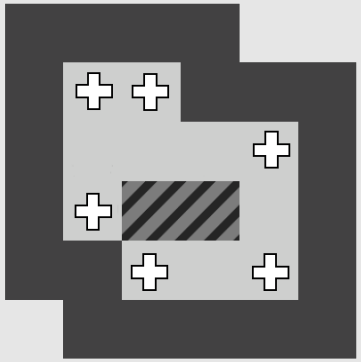
- **Le sous-arbre formé à partir de cet état est donc inutile** et doit être détecté et supprimé au plus vite.

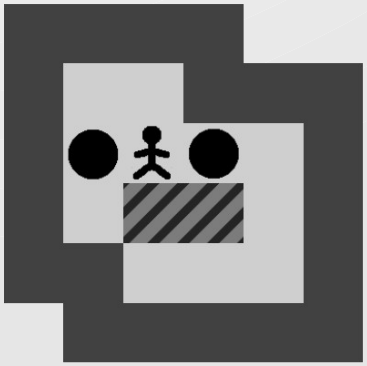
> Deadlock à une caisse

- **Deux cas** de deadlock provoqués par une seule caisse :

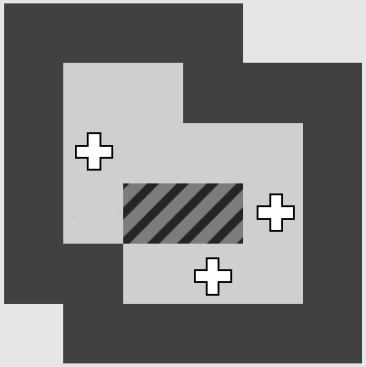


Deadlock en coin



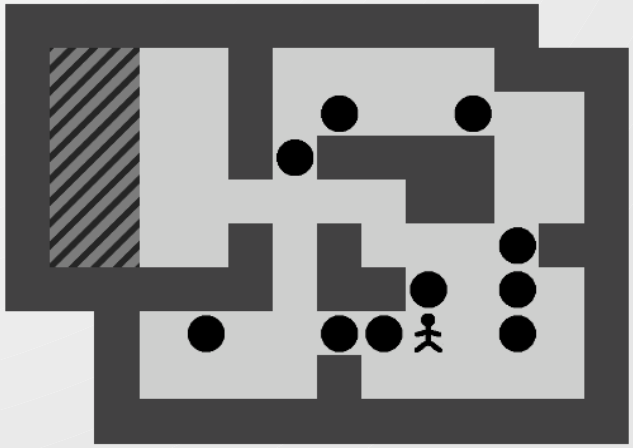


Deadlock en ligne

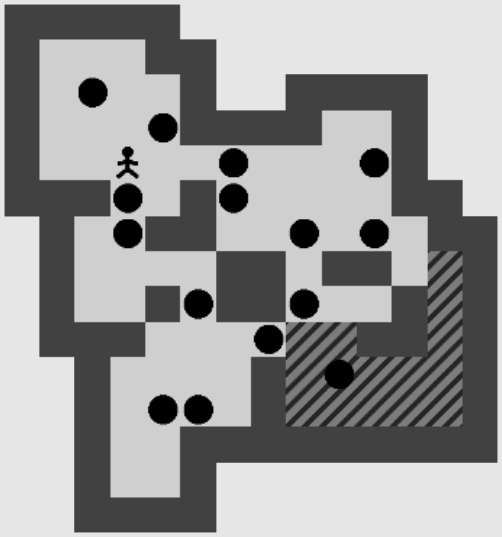


> Deadlock à plusieurs caisses

- Deadlock induit par la **dernière poussée** :



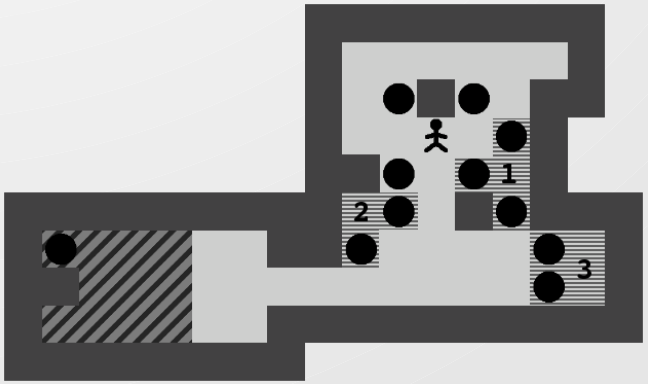
Deadlock en carré

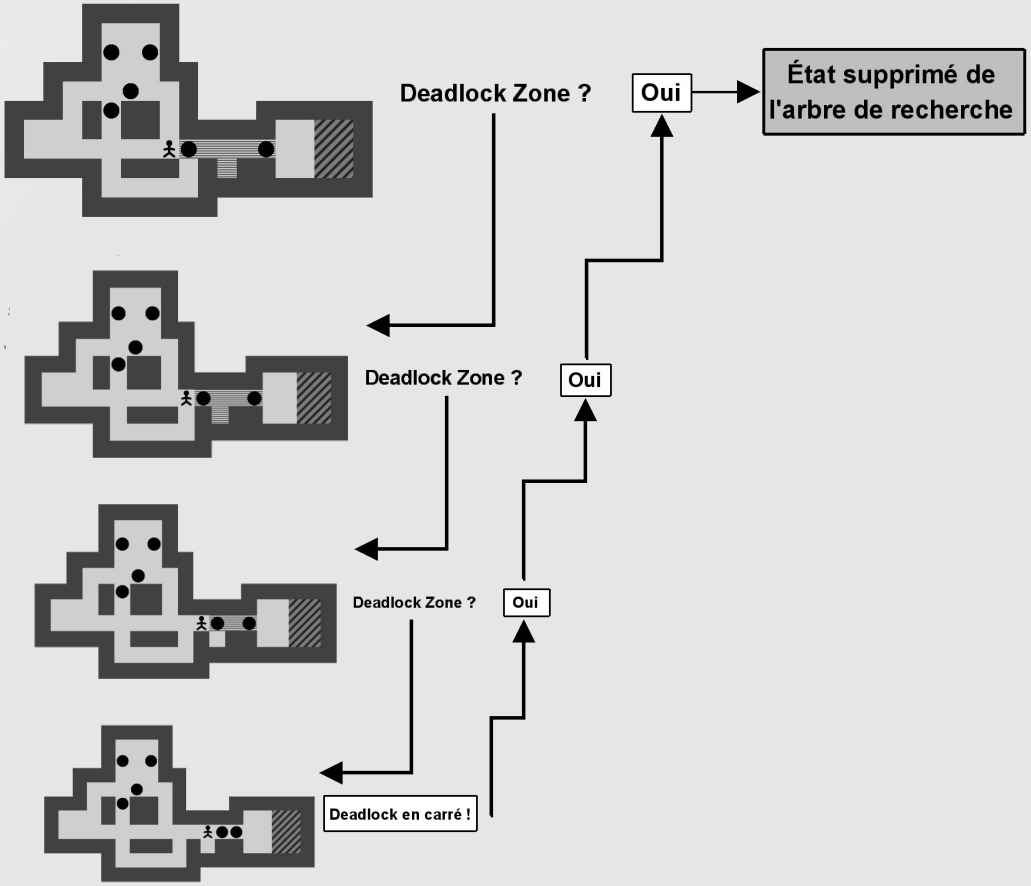


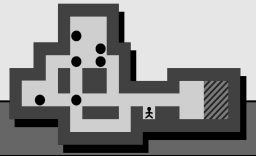
Deadlock en Z

> Deadlock à plusieurs caisses

- **Deadlock Zone** : sous-ensemble de positions dans lequel certaines caisses sont « bloquées »

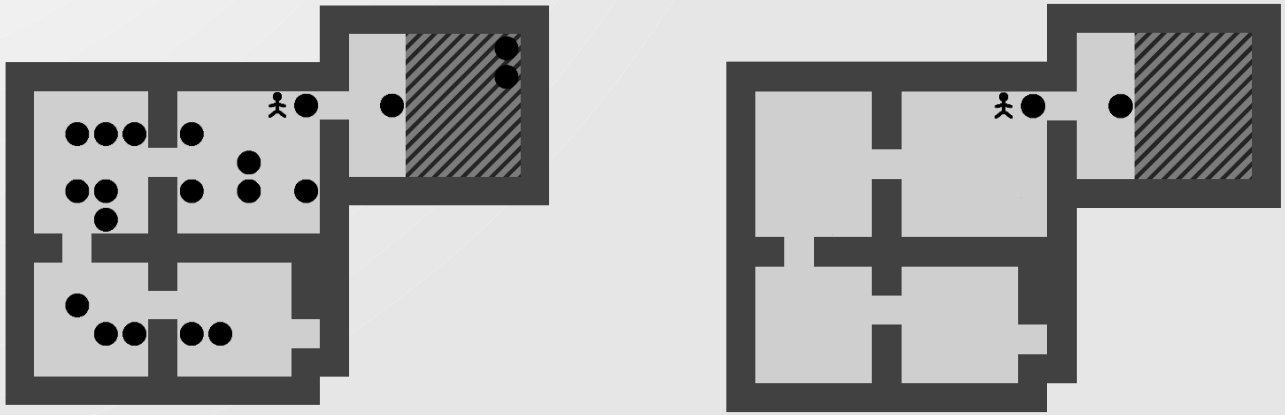




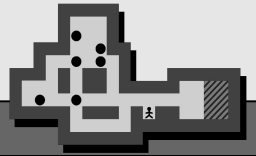


> Deadlock à plusieurs caisses

- **Deadlock méthodique** : recherche méthodique des *sous-états de 3 caisses et moins* qui provoquent des deadlocks non détectés par les méthodes précédentes.

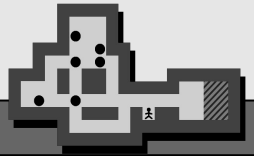


État deadlock provoqué par un sous-état formé de 2 caisses



> Structure de l'exposé

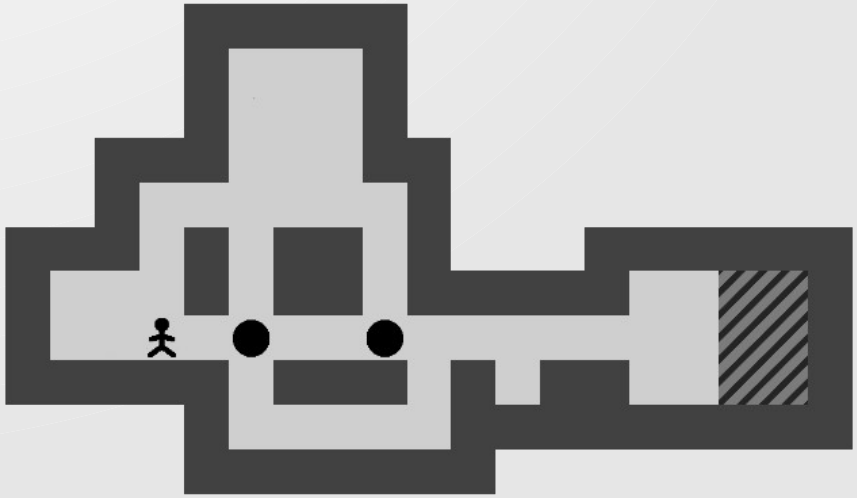
- **Sokoban**
  - Présentation
  - Règles
- **Résolution**
  - État de l'art
  - Arbre de recherche
- **Parcours**
  - en largeur
  - en profondeur
  - A*
  - IDA*
- **Estimation**
- **Deadlock**
  - À une caisse
  - À plusieurs caisses
- **Pénalité**
  - Définition
  - Recherche
- **Pré-traitement & Post-traitement**
- **Conclusion**



> Définition

- **Pénalité** : *sous-état* qui provoque un surcoût de l'estimation h(n) précédemment trouvée.

- Cette méthode permet d'**ajuster la valeur de h(n)** d'un état en fonction du surcoût occasionné par **plusieurs caisses qui se gênent mutuellement**

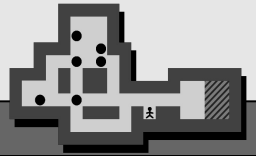


Solution optimale : 22 poussées

Estimation h(n) : 20 poussées

Pénalité du sous-état : 2

Sous-état qui provoque une pénalité de 2

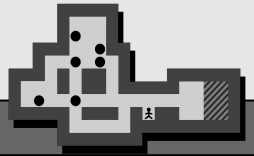


> Recherche

- Il existe **deux méthodes** pour rechercher les sous-états pénalisés :
  - **La recherche passive** : consiste à tester tous les sous-états de 3 caisses et moins dans un niveau pour voir si ils impliquent une pénalité.
  - **La recherche active** : lors du parcours de l'arbre, teste différents sous-états créés à partir des caisses d'un état intermédiaire.

> Structure de l'exposé

- **Sokoban**
  - Présentation
  - Règles
- **Résolution**
  - État de l'art
  - Arbre de recherche
- **Parcours**
  - en largeur
  - en profondeur
  - A*
  - IDA*
- **Estimation**
- **Deadlock**
  - À une caisse
  - À plusieurs caisses
- **Pénalité**
  - Définition
  - Recherche
- **Pré-traitement & Post-traitement**
- **Conclusion**



> Définitions

- **Pré-traitement** : travail effectué par le solveur sur un niveau **avant** de commencer à construire l'arbre de recherche.

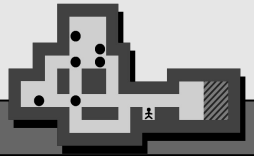
Permet de contraindre le solveur à trouver **tous les sous-états pénalisés de *n* caisses** avant de commencer la construction de l'arbre de recherche

- **Post-traitement** : travail effectué par le solveur sur un niveau **après** avoir construit et parcouru l'arbre de recherche sans trouver de solution.

Permet de détecter les positions du niveau les plus souvent occupées par des caisses. Celles-ci indiquent souvent des **deadlocks ou pénalités non détectées**

> Structure de l'exposé

- **Sokoban**
  - Présentation
  - Règles
- **Résolution**
  - État de l'art
  - Arbre de recherche
- **Parcours**
  - en largeur
  - en profondeur
  - A*
  - IDA*
- **Estimation**
- **Deadlock**
  - À une caisse
  - À plusieurs caisses
- **Pénalité**
  - Définition
  - Recherche
- **Pré-traitement & Post-traitement**
- **Conclusion**



> Conclusion

- Notre solveur a actuellement réussi à trouver **23 solutions optimales** sur l'ensemble testé de **90 niveaux**.

- Avec **plus de temps** pour le **pré-traitement** et le **parcours**, le solveur devrait être capable de trouver quelques solutions supplémentaires.

- Le solveur intègre de nouvelles méthodes inédites telles que les **deadlock zones**, **deadlocks en Z** et le **post-traitement**.

- Il existe des possibilités d'**évolution**. La plus prometteuse serait d'intégrer les **puissantes heuristiques de Talking Stone** pour trouver des **solutions non-optimales** lors de notre parcours de l'arbre de recherche.

> Questions

Merci de votre attention

Si vous avez des questions, n'hésitez pas à les poser